

Func

函数

函数的声明

编程中的函数（function）一般是若干语句的集合。我们也可以将其称作「子过程（subroutine）」。在编程中，如果有一些重复的过程，我们可以将其提取出来，形成一个函数。函数可以接收若干值，这叫做函数的参数。函数也可以返回某个值，这叫做函数的返回值。

声明一个函数，我们需要返回值类型、函数的名称，以及参数列表。

```
1 // 返回值类型 int
2 // 函数的名称 some_function
3 // 参数列表 int, int
4 int some_function(int, int);
```

如上图，我们声明了一个名为 `some_function` 的函数，它需要接收两个 `int` 类型的参数，返回值类型也为 `int`。可以认为，这个函数将会对传入的两个整数进行一些操作，并且返回一个同样类型的结果。

实现函数：编写函数的定义

只有函数的声明（declaration）还不够，他只能让我们在调用时能够得知函数的 **接口** 类型（即接收什么数据、返回什么数据），但其缺乏具体的内部实现，也就是函数的 **定义**（definition）。我们可以在 **声明之后的其他地方** 编写代码 **实现**（implement）这个函数（也可以在另外的文件中实现，但是需要将分别编译后的文件在链接时一并给出）。

如果函数有返回值，则需要通过 `return` 语句，将值返回给调用方。函数一旦执行到 `return` 语句，则直接结束当前函数，不再执行后续的语句。

```
1 int some_function(int, int); // 声明
2
3 /* some other code here... */
4
5 int some_function(int x, int y) { // 定义
6     int result = 2 * x + y;
7     return result;
8     result = 3; // 这条语句不会被执行
9 }
```

在定义时，我们给函数的参数列表的变量起了名字。这样，我们便可以在函数定义中使用这些变量了。

如果是同一个文件中，我们也可以直接将 **声明和定义合并在一起**，换句话说，也就是在声明时就完成定义。

```
1 int some_function(int x, int y) { return 2 * x + y; }
```

如果函数不需要有返回值，则将函数的返回值类型标为 `void`；如果函数不需要参数，则可以将参数列表置空。同样，无返回值的函数执行到 `return;` 语句也会结束执行。

```

1 void say_hello() {
2     cout << "hello!\n";
3     cout << "hello!\n";
4     cout << "hello!\n";
5     return;
6     cout << "hello!\n";    // 这条语句不会被执行
7 }

```

函数的调用

和变量一样，函数需要先被声明，才能使用。使用函数的行为，叫做「调用（call）」。我们可以在任何函数内部调用其他函数，包括这个函数自身。函数调用自身的行为，称为 **递归（recursion）**。

在大多数语言中，调用函数的写法，是 **函数名称加上一对括号（）**，如 `foo()`。如果函数需要参数，则我们将其需要的参数按顺序填写在括号中，以逗号间隔，如 `foo(1, 2)`。函数的调用也是一个表达式，**函数的返回值就是表达式的值**。

函数声明时候写出的参数，可以理解为在函数 **当前次调用的内部** 可以使用的变量，这些变量的值由调用处传入的值初始化。看下面这个例子：

```

1 void foo(int, int);
2
3 /* ... */
4
5 void foo(int x, int y) {
6     x = x * 2;
7     y = y + 3;
8 }
9
10 /* ... */
11
12 a = 1;
13 b = 1;
14 // 调用前: a = 1, b = 1
15 foo(a, b);    // 调用 foo
16                // 调用后: a = 1, b = 1

```

在上面的例子中，`foo(a, b)` 是一次对 `foo` 的调用。调用时，`foo` 中的 `x` 和 `y` 变量，分别由调用处 `a` 和 `b` 的值初始化。因此，在 `foo` 中对变量 `x` 和 `y` 的修改，**并不会影响到调用处的变量的值**。

如果我们需要在函数（子过程）中修改变量的值，则需要采用「传引用」的方式。

```

1 void foo(int& x, int& y) {
2     x = x * 2;
3     y = y + 3;
4 }
5
6 /* ... */
7
8 a = 1;
9 b = 1;
10 // 调用前: a = 1, b = 1
11 foo(a, b);    // 调用 foo
12                // 调用后: a = 2, b = 4

```

上述代码中，我们看到函数参数列表中的「`int`」后面添加了一个「`&`（and 符号）」，这表示对于 `int` 类型的**引用（reference）**。在调用 `foo` 时，调用处 `a` 和 `b` 变量分别初始化了 `foo` 中两个对 `int` 类型的引用 `x` 和 `y`。在 `foo` 中的 `x`

和 `y`，可以理解为调用处 `a` 和 `b` 变量的「别名」，即 `foo` 中对 `x` 和 `y` 的操作，就是对调用处 `a` 和 `b` 的操作。

main 函数

特别的，每个 C/C++ 程序都需要有一个名为 `main` 的函数。任何程序都将从 `main` 函数开始运行。

`main` 函数也可以有参数，通过 `main` 函数的参数，我们可以获得外界传给这个程序的指令（也就是「命令行参数」），以便做出不同的反应。

下面是一段调用了函数（子过程）的代码：

```
1 // hello_subroutine.cpp
2
3 #include <iostream>
4
5 void say_hello() {
6     std::cout << "hello!\n";
7     std::cout << "hello!\n";
8     std::cout << "hello!\n";
9 }
10
11 int main() {
12     say_hello();
13     say_hello();
14 }
```

本页面最近更新：2023/3/22 15:46:23, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [yang-lile](#), [CCXXxi](#), [lr1d](#), [isdanni](#), [kLiHz](#), [tsagaanbar](#), [Xeonacid](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用